

■ Viva Voce Preparation Guide

Node.js, Express.js & Backend Development
Comprehensive Question Bank — All Topics Covered

1. Middleware in Express.js

Q1.

What is middleware in Express.js? Explain its role in the request-response cycle.

Q2.

Describe the middleware lifecycle. What are the parameters of a middleware function?

Q3.

What is the difference between application-level and router-level middleware? Give examples of each.

Q4.

What is error-handling middleware? How does it differ from regular middleware in its function signature?

Q5.

What happens if you forget to call `next()` inside a middleware function?

Q6.

Can you stack multiple middleware functions on a single route? How?

Q7.

Name any three popular third-party middleware packages and explain what each does.

Q8.

What is Body Parser middleware? Why is it needed in Express.js?

Q9.

How does a request travel through an Express application from receipt to response?

Q10.

What is the difference between blocking and non-blocking code? Give an example of each.

Q11.

How does the non-blocking I/O model of Node.js relate to Express middleware?

Q12.

How would you write a logging middleware from scratch?

Q13.

What does `express.json()` do, and how is it different from `body-parser`?

Q14.

Can middleware be applied only to specific routes? Show how.

Q15.

What is the significance of the order in which middleware is defined in Express?

■ Tip: Always remember: middleware runs in the ORDER it is defined. The error-handler must have 4 arguments: (err, req, res, next).

2. SSR vs CSR & Template Engines

Q1.

What is the difference between Server-Side Rendering (SSR) and Client-Side Rendering (CSR)?

Q2.

What are the advantages of SSR over CSR in terms of SEO and performance?

Q3.

When would you prefer CSR over SSR? Give a real-world use case.

Q4.

What is a template engine? Why do we use template engines in Node.js?

Q5.

What is EJS? How do you set it up in a Node.js/Express application?

Q6.

What are the EJS tags? Explain `<% %>`, `<%= %>`, and `<%- %>` with examples.

Q7.

What is Handlebars (HBS)? How does it differ from EJS?

Q8.

How do you pass data from Express route to an EJS template?

Q9.

What are EJS partials? How do you include a partial in a template?

Q10.

How do you set a default template engine in Express?

Q11.

What is the `res.render()` method? What arguments does it accept?

Q12.

What is the `views` directory in Express? How can you change its default path?

Q13.

What are Handlebars helpers? How do you create a custom helper?

Q14.

Explain the concept of layouts in HBS.

Q15.

Compare EJS, HBS, and Pug — when would you choose one over the others?

■ Tip: Be ready to write a small EJS snippet on the board. Know all three tag types by heart.

3. Databases — SQL, NoSQL & MongoDB

Q1.

What is a database? Distinguish between SQL and NoSQL databases.

Q2.

What are the main types of NoSQL databases? Give examples of each.

Q3.

When should you choose MongoDB over a relational database like MySQL?

Q4.

What is MongoDB? How does it store data?

Q5.

Explain the terms: collection, document, field, and BSON.

Q6.

How do you connect MongoDB to a Node.js application using Mongoose?

Q7.

What is Mongoose? What is an ODM and how does Mongoose act as one?

Q8.

What is a Mongoose Schema? What is a Mongoose Model?

Q9.

What are the common Mongoose data types? Give examples.

Q10.

How do you perform CRUD operations using Mongoose?

Q11.

What are Mongoose validators? Give two examples.

Q12.

What is the difference between `find()` and `findOne()` in Mongoose?

Q13.

What is `populate()` in Mongoose? How does it handle references between collections?

Q14.

What are Mongoose middleware (pre/post hooks)? Give a use case.

Q15.

What is the difference between `save()` and `insertMany()` in Mongoose?

Q16.

What are indexes in MongoDB and why are they important?

Q17.

What is the difference between `mongoose.connect()` and `mongoose.createConnection()`?

■ Tip: Know how to write a full Mongoose Schema with at least one reference/populate. CRUD using `async/await` is commonly tested.

4. Session Management — Cookies & Express-Sessions

Q1.

What is a cookie? How is it different from a session?

Q2.

How do you set and read cookies in Express.js?

Q3.

What is the `cookie-parser` middleware? How is it used?

Q4.

What are the security flags for cookies? Explain `HttpOnly`, `Secure`, and `SameSite`.

Q5.

What is a session? How does Express-session work under the hood?

Q6.

What does the `secret` option in `express-session` do?

Q7.

What is the difference between a session store and a cookie store?

Q8.

How do you configure MongoDB as a session store (e.g., connect-mongo)?

Q9.

What is the resave and saveUninitialized option in express-session?

Q10.

How do you destroy a session in Express? When is this useful?

Q11.

What is the difference between req.session and req.cookies?

Q12.

What are signed cookies? How do they differ from unsigned cookies?

Q13.

How can sessions be used to implement a flash message system?

Q14.

What are the security risks of poor session management?

Q15.

What is a session ID? Where is it stored?

■ **Tip: Session vs Cookie is a favourite question. Remember: the session data lives on the SERVER; the cookie holds only the session ID.**

5. Authentication — Bcrypt, JWT & Passport.js

Q1.

What is authentication? How is it different from authorisation?

Q2.

What is Bcrypt? Why should you never store plain-text passwords?

Q3.

Explain the concept of salting in Bcrypt. What does the salt round number mean?

Q4.

What is the difference between bcrypt.hash() and bcrypt.compare()?

Q5.

What is a JWT (JSON Web Token)? What are its three parts?

Q6.

How does JWT authentication work? Describe the full flow.

Q7.

What information is typically stored in a JWT payload?

Q8.

What is the difference between signing and encrypting a JWT?

Q9.

What are the advantages of JWT over session-based authentication?

Q10.

What is token expiry in JWT? How do you handle token refresh?

Q11.

Where should you store a JWT on the client side? What are the trade-offs?

Q12.

What is Passport.js? Why is it used in Express applications?

Q13.

What is a Passport strategy? Name two built-in or popular strategies.

Q14.

Explain the use of `passport.authenticate()` middleware.

Q15.

What do `serializeUser` and `deserializeUser` do in Passport.js?

Q16.

How do you implement Google OAuth with Passport.js?

Q17.

What is the difference between local strategy and OAuth strategy in Passport?

Q18.

What is the difference between stateful (session) and stateless (JWT) authentication?

■ Tip: Be prepared to draw the JWT flow on the board: Login → Server signs token → Client stores → Client sends in header → Server verifies.

6. Full-Duplex Communication & Socket.io

Q1.

What is full-duplex communication? How does it differ from half-duplex?

Q2.

What are WebSockets? How do they differ from HTTP?

Q3.

What is the WebSocket handshake process?

Q4.

What is Socket.io? What problem does it solve over raw WebSockets?

Q5.

What is long polling? How does Socket.io fall back to it?

Q6.

How do you set up a basic Socket.io server with Express?

Q7.

How do you connect a client to a Socket.io server?

Q8.

What are Socket.io events? How do you emit and listen to events?

Q9.

What is the difference between `socket.emit()` and `io.emit()`?

Q10.

What are Socket.io rooms? How do you join and leave a room?

Q11.

What is the difference between `broadcast.emit()` and `io.to(room).emit()`?

Q12.

How do you handle the disconnect event in Socket.io?

Q13.

What are namespaces in Socket.io? When would you use them?

Q14.

How does Socket.io handle reconnection automatically?

Q15.

How would you build a real-time chat application using Socket.io?

Q16.

What is the difference between `socket.on()` and `io.on()`?

■ Tip: Draw a simple diagram of how `io.emit`, `socket.emit`, and `socket.broadcast.emit` differ in who receives the message.

7. PostgreSQL

Q1.

What is PostgreSQL? How does it differ from MySQL?

Q2.

What is the difference between PostgreSQL and MongoDB?

Q3.

How do you connect PostgreSQL with a Node.js application? Which packages are used?

Q4.

What is the `pg` (node-postgres) library? How do you execute a query with it?

Q5.

What is a connection pool? Why is it important in PostgreSQL with Node.js?

Q6.

What is an ORM? How does Sequelize work with PostgreSQL?

Q7.

How do you write a basic `SELECT`, `INSERT`, `UPDATE`, `DELETE` query in PostgreSQL?

Q8.

What are PostgreSQL data types? Name at least five.

Q9.

What is a `PRIMARY KEY` and `FOREIGN KEY` in PostgreSQL?

Q10.

What are `JOINS` in SQL? Explain `INNER JOIN`, `LEFT JOIN`, and `RIGHT JOIN`.

Q11.

What is a transaction in PostgreSQL? How do you implement it in Node.js?

Q12.

What is the difference between `WHERE` and `HAVING` clause?

Q13.

What are PostgreSQL indexes? Why are they used?

Q14.

What is normalization? Explain 1NF, 2NF, 3NF briefly.

Q15.

How do you handle parameterised queries to prevent SQL injection?

Q16.

What is the difference between `TRUNCATE` and `DELETE` in PostgreSQL?

■ Tip: Know how to write a simple parameterised query with the `pg` library: `pool.query('SELECT * FROM users WHERE id = $1', [id]).`

■ Quick Revision — Key Concepts at a Glance

Concept	One-Line Definition
Middleware	Function with (req, res, next) that processes request in pipeline
Body Parser	Parses incoming request body (JSON/URL-encoded)
SSR	HTML generated on server; sent to browser
CSR	HTML generated in browser using JavaScript
EJS Tag <%=	Outputs escaped HTML value
EJS Tag <%-	Outputs unescaped (raw) HTML value
Mongoose Schema	Blueprint defining structure of a MongoDB document
populate()	Replaces ObjectId references with actual documents
Cookie	Small data stored on client browser
Session	Server-side storage; only session ID sent to client
Bcrypt salt	Random data added before hashing to prevent rainbow attacks
JWT	Self-contained token with Header.Payload.Signature
Passport Strategy	Plugin that defines how to authenticate (local, OAuth, JWT)
Socket.io room	Named channel for grouping sockets together
io.emit()	Sends event to ALL connected clients
socket.broadcast.emit()	Sends to all EXCEPT the sender
Connection Pool (PG)	Reuses DB connections to improve performance
SQL Injection	Prevented by parameterised queries (\$1, \$2 placeholders)
Transaction	Group of queries that succeed or fail together (ACID)

Best of luck for your Viva! Stay calm, think before you answer, and if unsure — explain your reasoning.
You've got this! ■